

Arreglos Dinámicos

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo



Arreglos Dinámicos

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo

Objetivo General

Comprender qué son los arreglos dinámicos en C++, cómo se declaran, almacenan y liberan en memoria, diferenciando su uso respecto a los arreglos estáticos, mediante ejemplos prácticos y el manejo de memoria con new y delete.

Objetivos Específicos

- Diferenciar entre arreglos estáticos y dinámicos en cuanto a su declaración, asignación y uso en C++.
- Aplicar la sintaxis de creación y liberación de memoria dinámica utilizando punteros y los operadores new y delete para manipular arreglos de tamaño variable.

- **La memoria automática** (asociada al stack) es un tipo de almacenamiento donde el espacio de memoria se asigna al entrar a una función y se libera automáticamente al salir.
- Se utiliza para ***variables locales y arreglos de tamaño fijo.***
- El tamaño debe ser conocido en tiempo de compilación.
- **Desventaja:** no se puede cambiar su tamaño durante la ejecución del programa.

Uso de memoria dinámica

■ STACK (Memoria Estática)

Aquí se almacenan las variables locales del programa, como `int n = 3;` y el puntero `int* p;`

En este ejemplo, n guarda el número de elementos que el usuario desea, es decir: entrada = 3.

El puntero p es una variable en el stack que apunta a una dirección en el heap.

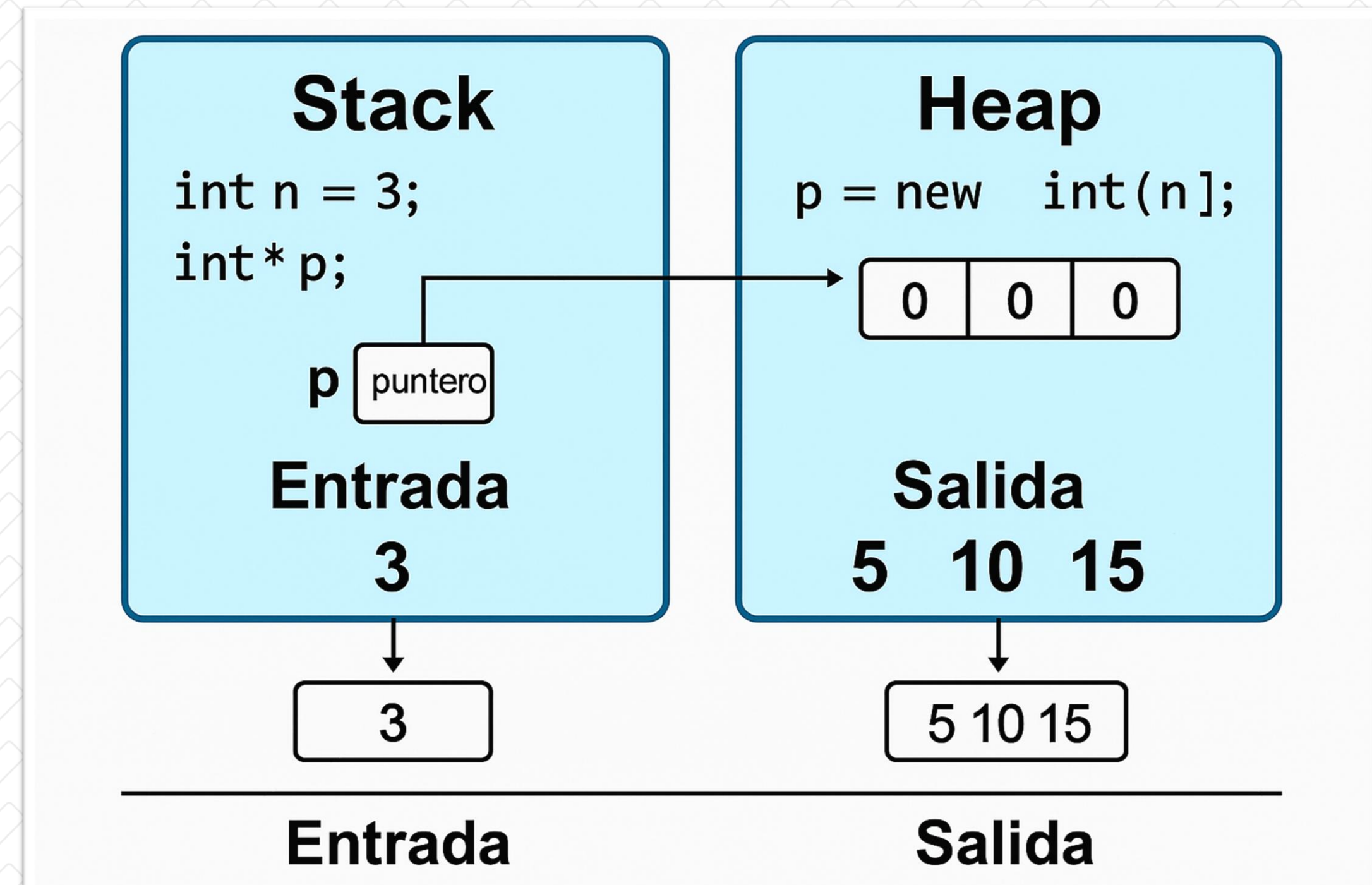
■ HEAP (Memoria Dinámica)

Aquí se crea el arreglo dinámico con new:

`p = new int(n);`

Esto significa: “Reserva espacio en el heap para guardar 3 enteros”. En la imagen, inicialmente el heap contiene: [0][0][0].

Luego, se llenan esos espacios con los valores ingresados o calculados:



🔄 Relación visual entre Stack y Heap

- El puntero `p` que vive en el stack tiene una flecha que apunta al arreglo que vive en el heap.
- Esto ilustra cómo un puntero conecta ambos espacios de memoria.
- Por eso decimos que el stack “controla” el acceso al heap.

“En programación, saber cómo administrar memoria es tan importante como saber cómo escribir una variable.”

Un arreglo dinámico es un tipo de arreglo **cuyo tamaño no se conoce al momento de escribir el programa**, y se define durante la ejecución (en tiempo de ejecución).

A diferencia de los arreglos estáticos (que tienen tamaño fijo), los dinámicos permiten que el usuario elija el tamaño mientras el programa se está ejecutando.

Esto se logra utilizando:

- Punteros (int*)
- El operador new para reservar espacio en memoria
- El operador delete para liberar ese espacio cuando ya no se necesita



¿Y por qué usar punteros en arreglos dinámicos entonces?

Cuando **no sabemos el tamaño máximo posible**, o este puede ser **muy grande**, no conviene reservar memoria fija por adelantado.

Entonces, usamos **punteros y new** para **crear el arreglo en tiempo de ejecución con el tamaño exacto que se necesite**, sin desperdiciar espacio.

Arreglo dinámicos

🔍 ¿Cuál es la función del puntero en un arreglo dinámico?

El puntero sirve para guardar la dirección en memoria donde se almacenará el arreglo dinámico.

Cuando usamos new, el sistema reserva espacio en el heap y devuelve una dirección de memoria.

Ese valor se debe guardar en un puntero, ya que las variables normales no pueden manejar direcciones directamente.

“El heap es como una bodega donde puedes pedir espacio cuando lo necesitas y devolverlo cuando termines de usarlo.”

📌 Características del heap:

✅ Ventajas

Tamaño flexible en tiempo de ejecución

Útil para arreglos grandes o variables

Vive más allá del final de una función

⚠️ Consideraciones

Tú eres responsable de liberar la memoria

Si no liberas, puedes causar “fugas de memoria”

Usar `new` y `delete` correctamente es crucial


```
main.cpp
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int tamano;
6
7      // Solicitar al usuario el tamaño del arreglo
8      cout << "Ingrese el tamaño del arreglo: ";
9      cin >> tamano;
10
11     // Crear el arreglo dinámico en el heap usando 'new'
12     int* arreglo = new int[tamano];
13
14     // Llenar el arreglo con datos ingresados por el usuario
15     cout << "\nIngrese " << tamano << " números:\n";
16     for (int i = 0; i < tamano; i++) {
17         cout << "Elemento [" << i << "]: ";
18         cin >> arreglo[i];
19     }
20
21     // Mostrar el contenido del arreglo
22     cout << "\nContenido del arreglo:\n";
23     for (int i = 0; i < tamano; i++) {
24         cout << "Elemento [" << i << "] = " << arreglo[i] << endl;
25     }
26
27     // Liberar la memoria que se reservó con new
28     delete[] arreglo;
29
30     // Avisar que la memoria fue liberada
31     cout << "\nMemoria liberada con delete[].\n";
32
33     return 0;
34 }
```

Explicación paso a paso

Línea(s)	Acción
int tamano;	Se declara una variable para que el usuario elija el tamaño del arreglo.
int* arreglo = new int[tamano];	Se reserva memoria dinámica en el heap para guardar <code>tamano</code> elementos de tipo <code>int</code> .
for (...)	Se usa un ciclo para llenar el arreglo con valores ingresados por el usuario.
Segundo for (...)	Se usa otro ciclo para imprimir el contenido del arreglo.
delete[] arreglo;	Se libera la memoria del heap para evitar fugas.
cout << ...	Se notifica que la memoria fue liberada correctamente.

Ingrese el tamaño del arreglo: 3

Ingrese 3 números:
Elemento [0]: 1
Elemento [1]: 2
Elemento [2]: 3

Contenido del arreglo:
Elemento [0] = 1
Elemento [1] = 2
Elemento [2] = 3

Memoria liberada con delete[].

"Dominar la memoria dinámica es como tener el control total del espacio en tu programa: tú decides cuánto, cuándo y cómo usarlo. Así se construye software inteligente."

Arreglos Dinámicos

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo

Arreglos Dinámicos

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo

